Advanced Computer Systems Class Wide Report

Ethan Katz

Table of Contents

Introduction

Scalar baseline vs. auto-vectorized build for each selected kernel across working-set sizes

- Bandwidth
- Runtime and Working-Set Analysis
- Speedup by Kernel

Locality Sweep Across Cache Levels

- Speedup by Elementwise Multiplication

Alignment & Tail-Handling Study

- Alignment Experiment
- Runtime vs Tail-Length

Stride/Gather Study

- Runtime vs Stride
- Speedup vs Stride

Data-Type Comparison

- Multiply Runtime
- Saxpy Runtime

Vectorization Verification

- Speedup vs Size
- Alignment Runtime

Roofline Interpretation for One Kernel

- Triad Kernel
- Scalar vs SIMD runtime

One Negative Example

- Triad Kernel Achieved Performance
- Roofline Analysis for Triad

Conclusion

Honest AI Usage

Introduction

SIMD (Single Instruction, Multiple Data) lets a processor run one instruction on several values at once. On this machine, AVX2 handles eight FP32 values per instruction. This report looks at when that actually speeds things up and when it doesn't.

Scalar baseline vs. auto-vectorized build for each selected kernel across working-set sizes

This section checks whether SIMD speeds up each kernel and how that changes with input size.

The three kernels are elementwise multiplication ($z_i = x_i y_i$), SAXPY ($y_i = ax_i + y_i$), and dot product ($s = \sum_i x_i y_i$). Each was tested at N = 1024, 8192, 65536, 1048576, and 8388608.

The scalar build uses:

```
wayslab@wayslab:~/simd-advantage-profiling$ g++ -O3 -fno-tree-vectorize -f
no-tree-slp-vectorize \
    src/benchmark_day2.cpp -o benchmark_day2_scalar
```

which disables GCC's loop and SLP vectorization.

The native-optimized build uses:

```
wayslab@wayslab:~/simd-advantage-profiling$ g++ -O3 -march=native \
    src/benchmark_day2.cpp -o benchmark_day2_simd
```

which allows all optimizations supported by the processor.

Bandwidth

Figure A compares the effective logical bandwidth of scalar and compiler-optimized builds from 2^{10} to 2^{23} FP32 elements. The Core Ultra 7 255U supports AVX2, which processes eight FP32 values per 256-bit instruction, but the actual gain depends on the kernel and input size.

Elementwise multiplication gains the most at small sizes. The gain shrinks as the working set grows, and the bump at the largest size is probably noise.

SAXPY also benefits, though less as the working set grows. Dot product barely improves, likely because its reduction doesn't vectorize well.

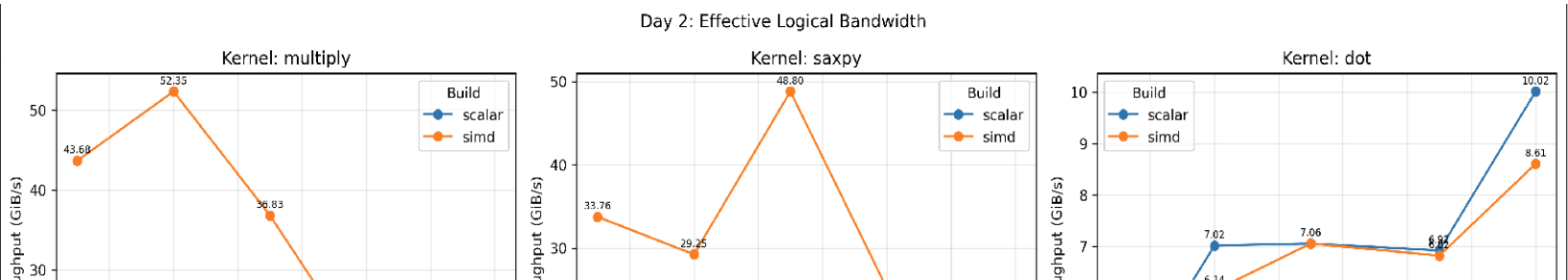


Figure A. Effective logical bandwidth for scalar and optimized builds across kernels and input sizes.

Runtime and Working-Set Analysis

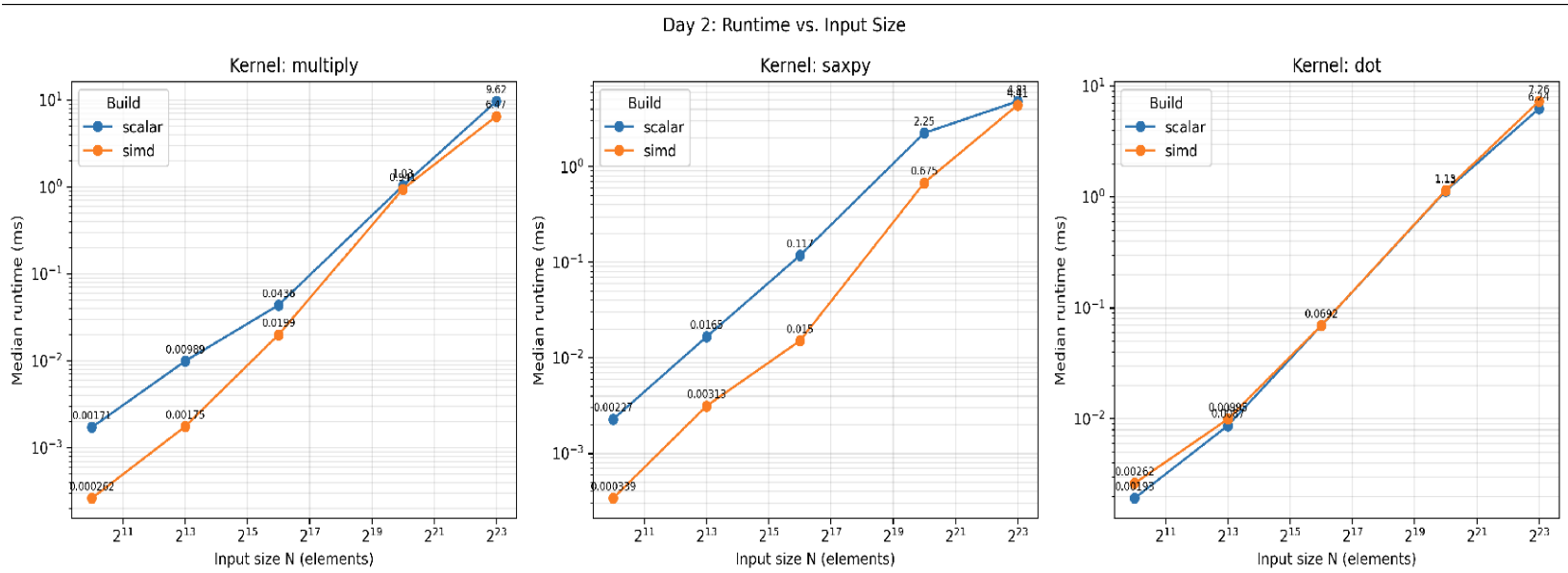
Figure B compares median execution time for both builds at 2^{10}, 2^{13}, 2^{16}, 2^{20}, and 2^{23} FP32 elements.

Elementwise multiplication runs in 1.71 µs (scalar) and 0.262 µs (optimized) at 2^{10}, a 6.54× speedup. At 2^{20} the runtimes are 1.03 ms and 0.941 ms, reducing the speedup to 1.09×.

SAXPY peaks at 2^{16}, with 0.117 ms (scalar) and 0.015 ms (optimized), a 7.80× speedup. The speedup falls to 3.33× at 2^{20} and 1.09× at 2^{23}.

Dot product shows little improvement. At 2^{10} the optimized build is slower than scalar (2.62 µs vs. 1.93 µs).

So multiply and SAXPY gain a lot at some sizes, and dot product doesn't.



Speedup by Kernel

Speedup is scalar runtime divided by optimized runtime, so anything above 1× means the optimized build is faster.

Elementwise multiplication reaches 6.54× at 2^{10} and 5.66× at 2^{13}, falls to 2.19× at 2^{16} and 1.09× at 2^{20}, then rises to 1.49× at 2^{23}.

SAXPY reaches the highest speedup, 7.80× at 2^{16}, before falling to 3.33× at 2^{20} and 1.09× at 2^{23}.

Dot product stays between 0.73× and 1.00×, so the optimized build gives little benefit and is sometimes slower.

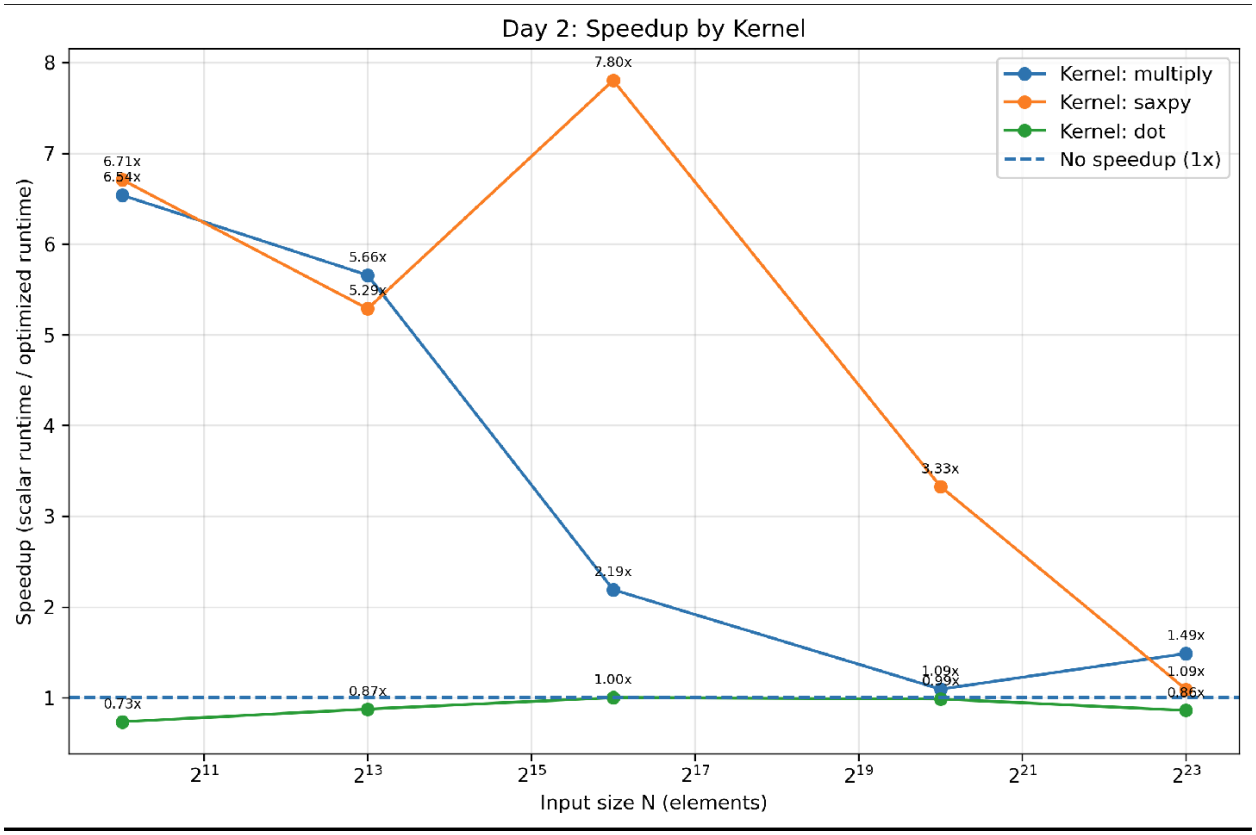


Figure C. Speedup by kernel (scalar runtime / optimized runtime) across input sizes.

Locality Sweep Across Cache Levels

This experiment looks at how working-set size affects SIMD speedup for elementwise multiplication, using the same two builds as above.

Speedup by Elementwise Multiplication

At 12 bytes of logical traffic per element (two reads, one write), the working sets range from 12 KiB to 96 MiB.

- **N = 2^{10} (12 KiB): 3.15× speedup.** The smallest workload benefits most.
- **N = 2^{13} (96 KiB): 2.37× speedup.** SIMD keeps a clear advantage.
- **N = 2^{16} (768 KiB): 2.43× speedup.** Speedup holds despite the larger working set.
- **N = 2^{20} (12 MiB): 1.52× speedup.** The SIMD advantage shrinks.
- **N = 2^{23} (96 MiB): 1.41× speedup.** The advantage shrinks further.

SIMD wins at every size, but the margin shrinks as the data outgrows the caches. Timing alone can't prove that's the cause, but it fits.

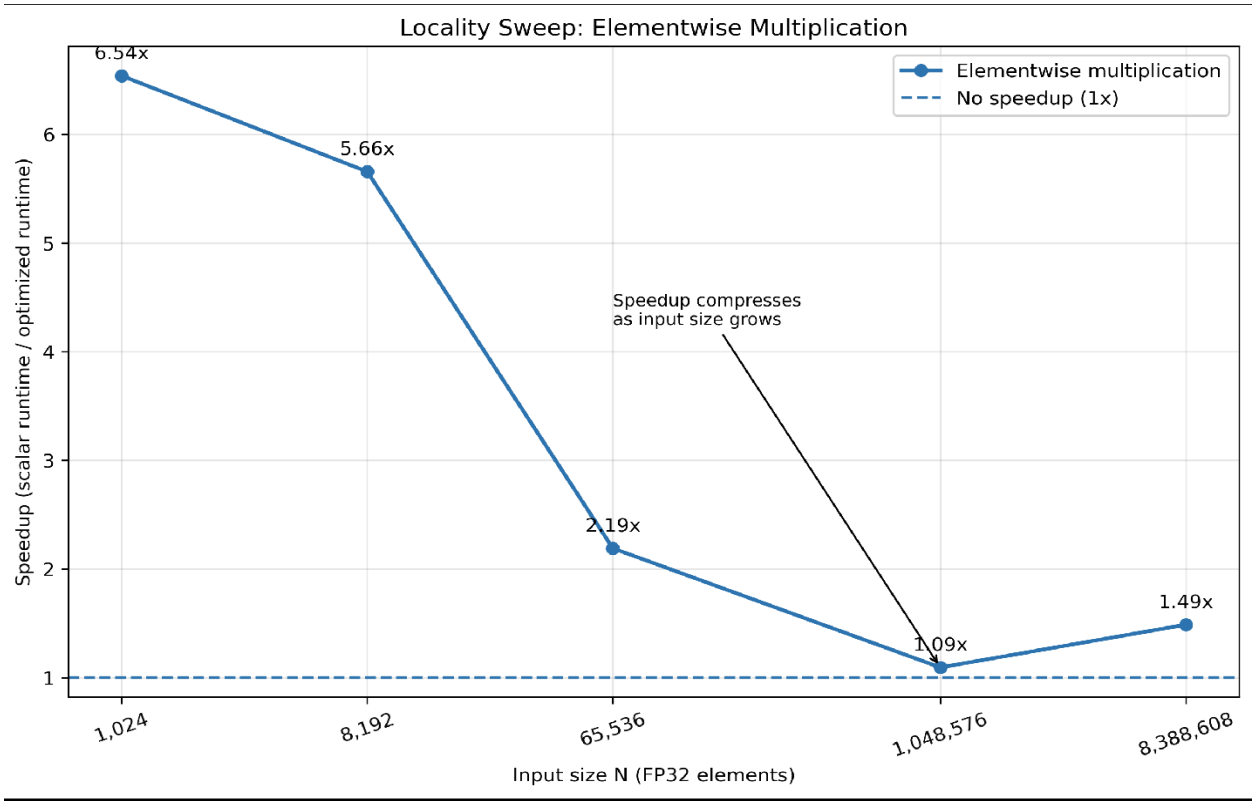


Figure D. Locality sweep: elementwise multiplication speedup across input sizes.

The test system reports a 48 KiB L1 data cache, 2 MiB L2, and 12 MiB L3, with 64-byte lines. The smallest workload fits in L1, the intermediate ones fit in L2, the 2^{20}-element workload matches L3, and the largest far exceeds it.

```
    linux-cloud-tools-oem
(.venv) wayslab@wayslab:~/simd-advantage-profiling$ /usr/lib/linux-tools/6.17.0-1033-oem/perf stat \
  -e cycles,instructions,cache-references,cache-misses \
  ./benchmark_scalar 1048576 /tmp/perf_scalar.csv
bash: /usr/lib/linux-tools/6.17.0-1033-oem/perf: No such file or directory
(.venv) wayslab@wayslab:~/simd-advantage-profiling$ lscpu -C
NAME ONE-SIZE ALL-SIZE WAYS TYPE        LEVEL  SETS PHY-LINE COHERENCY-SIZE
L1d       48K     352K   12 Data            1    64        1             64
L1i       64K     640K   16 Instruction     1    64        1             64
L2         2M      10M   16 Unified         2  2048        1             64
L3        12M      12M   12 Unified         3 16384        1             64
(.venv) wayslab@wayslab:~/simd-advantage-profiling$ for cache in /sys/devices/system/cpu/cpu0/cache/index*; do
    echo "Cache: $cache"
    printf "Level: "; cat "$cache/level"
    printf "Type: "; cat "$cache/type"
    printf "Size: "; cat "$cache/size"
    printf "Line size: "; cat "$cache/coherency_line_size"
    echo
done
Cache: /sys/devices/system/cpu/cpu0/cache/index0
Level: 1
Type: Data
Size: 48K
Line size: 64

Cache: /sys/devices/system/cpu/cpu0/cache/index1
Level: 1
Type: Instruction
Size: 64K
Line size: 64

Cache: /sys/devices/system/cpu/cpu0/cache/index2
Level: 2
Type: Unified
Size: 2048K
Line size: 64

Cache: /sys/devices/system/cpu/cpu0/cache/index3
Level: 3
Type: Unified
Size: 12288K
Line size: 64

(.venv) wayslab@wayslab:~/simd-advantage-profiling$
```

Figure E. Cache hierarchy reported by lscpu and sysfs on the test system.

Alignment & Tail-Handling Study

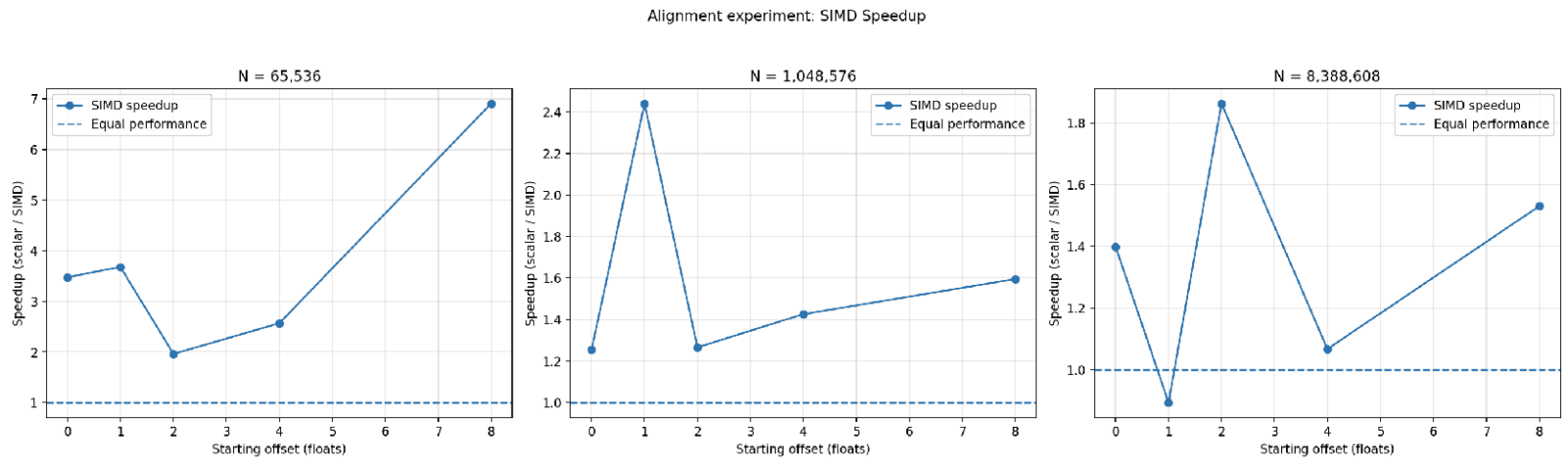
This experiment checks whether misaligned data or leftover "tail" elements slow down elementwise multiplication.

Alignment Experiment

Figure F compares scalar and SIMD runtimes at 65,536, 1,048,576, and 8,388,608 elements, with offsets of 0, 1, 2, 4, and 8 floats from a 64-byte-aligned allocation (working sets of 768 KiB to 96 MiB).

At 65,536 elements, SIMD runtimes range from 0.014 to 0.032 ms, and aligned and misaligned averages differ by about 3% excluding the outlier at offset 2. At 1,048,576 elements both average about 0.73 ms, and at 8,388,608 they differ by about 2%.

Neither build shows a clear alignment penalty. If there is a small one, it would take more repeated runs to see it.



Runtime vs Tail-Length

Figure G shows SIMD runtime as the array length grows from 2^{20} to $2^{20} + 7$ elements. Because the base length is a multiple of eight, this tests the cost of leftover elements at a vector-width boundary. The extra elements add negligible work, so no large runtime change is expected.

For remainders of 0 to 3, runtimes stay within 0.736 to 0.758 ms (about 3%). Runtime then drops to 0.675 ms at remainder 4 and rises to 1.083 ms at remainder 7, far more than a few extra elements can explain.

That jump is too big to blame on a few extra elements; noise, clock-speed changes, or scheduling are more likely. So there's no measurable tail-handling penalty here.

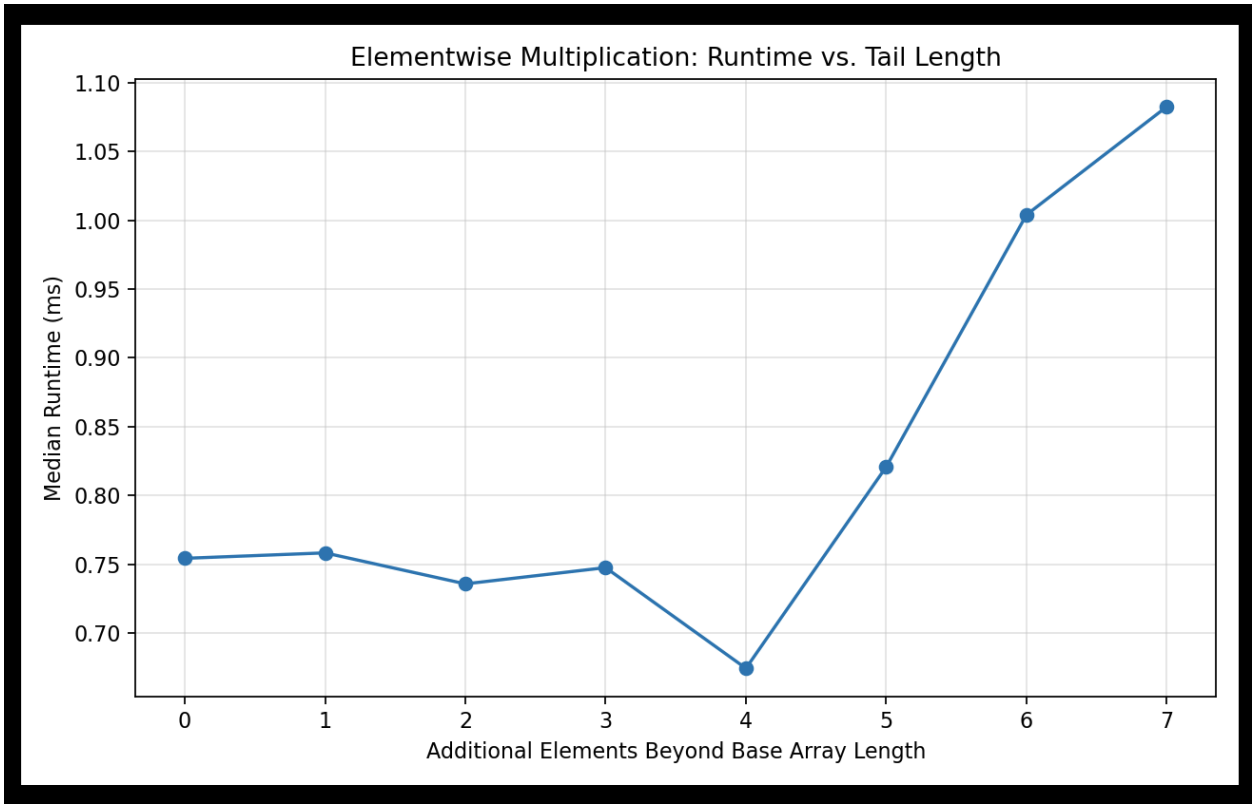


Figure G. Elementwise multiplication runtime vs. tail length.

Stride/Gather Study

This experiment tests what happens when elementwise multiplication skips through memory instead of reading it in order. At stride 1 every value in each cache line gets used; larger strides waste more of each line, so the question is whether SIMD keeps its advantage.

Runtime vs Stride

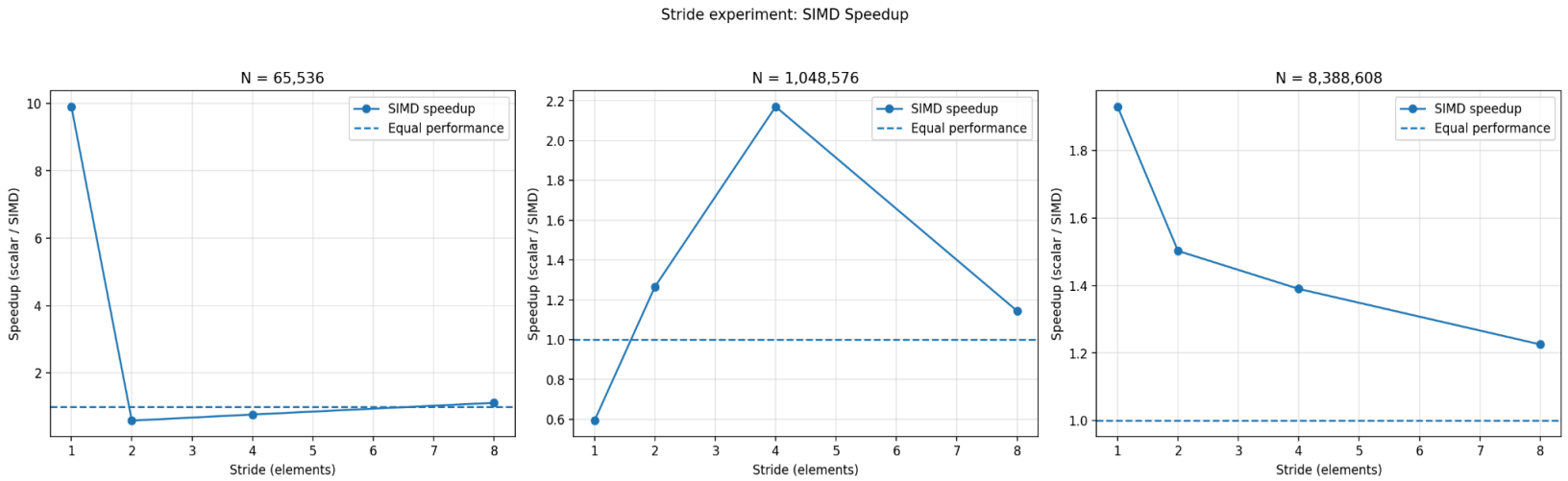
Figure H compares median scalar and SIMD runtimes at strides of 1, 2, 4, and 8 for three input sizes. Each iteration reads two elements and writes one.

At **N = 8,388,608**, runtime rises steadily with stride: SIMD from 5.75 ms to 10.1, 19.2, and 46.3 ms at strides 2, 4, and 8, and scalar from 11.1 ms to 15.2, 26.6, and 56.9 ms. The SIMD runtime at stride 8 is about eight times that at stride 1, consistent with reduced memory-access efficiency.

At **N = 1,048,576**, results are less consistent. SIMD at stride 1 takes 1.69 ms against 1.0 ms for scalar, and scalar runtimes at strides 4 and 8 are nearly identical (5.7 and 5.65 ms), so these points should be treated with caution.

At **N = 65,536**, SIMD is fastest at stride 1 (0.014 ms) and takes 0.104 to 0.167 ms at other strides. This suggests a benefit from contiguous access, though irregular scalar results make the cause hard to isolate.

Overall, bigger strides make things much slower, especially at the largest size, which is what you'd expect from worse locality.



Speedup vs Stride

Figure I shows SIMD speedup (scalar runtime divided by SIMD runtime).

At **N = 65,536**, speedup falls from 9.9× at stride 1 to 0.59×, 0.76×, and 1.11× at strides 2, 4, and 8. The 9.9× value exceeds the 8-lane limit and may partly reflect measurement variability. Assembly inspection would show whether non-unit strides use vector loads, scalar loads, or gathers.

At **N = 1,048,576**, speedup ranges from 0.59× (stride 1) to 2.17× (stride 4). This reflects the irregular runtimes in Figure H and shows no consistent trend.

At **N = 8,388,608**, speedup falls steadily from 1.93× at stride 1 to 1.50×, 1.39×, and 1.23× at strides 2, 4, and 8. SIMD stays faster, but its advantage narrows as memory-access costs grow.

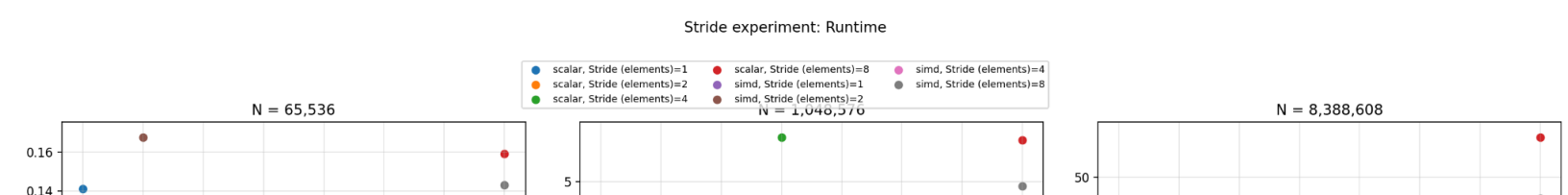


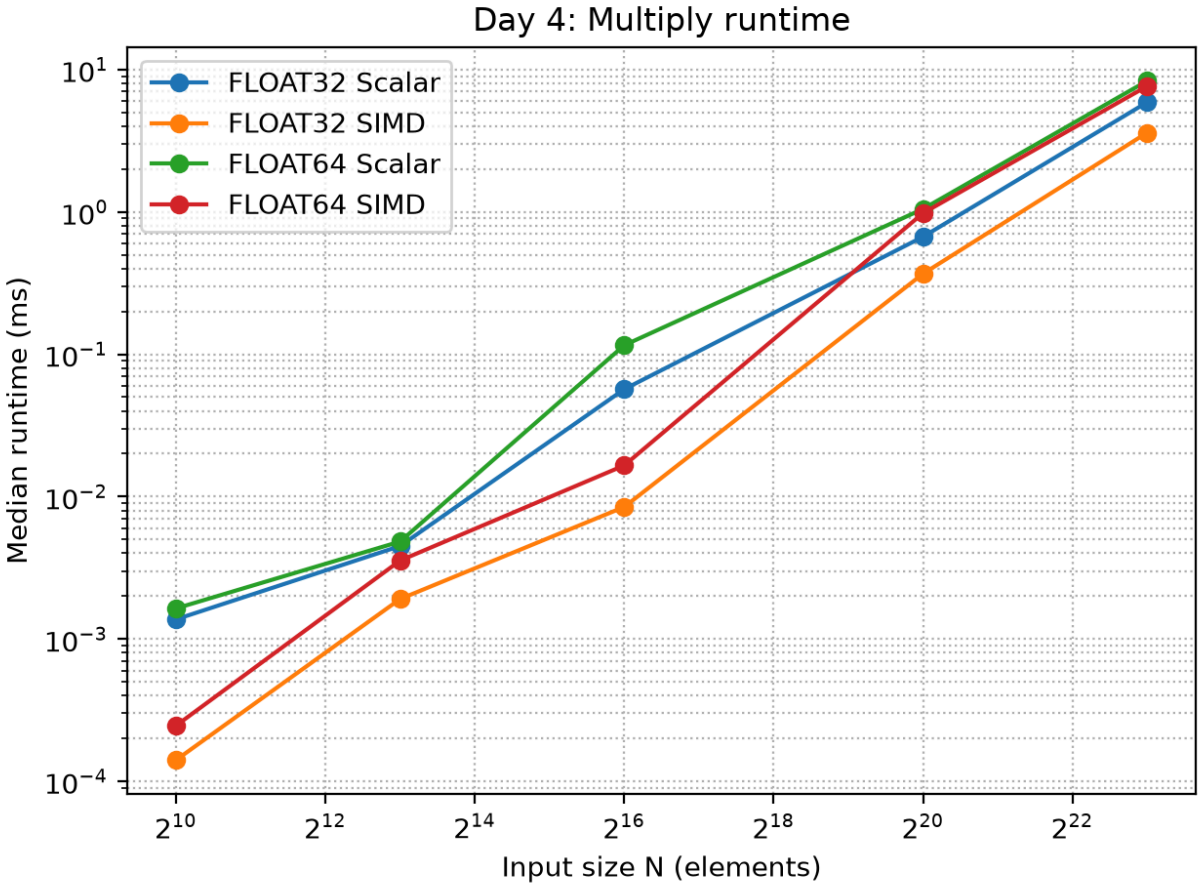
Figure I. SIMD speedup vs. stride for three input sizes.

Data-Type Comparison

This experiment compares single-precision (FP32) and double-precision (FP64) performance for scalar and SIMD builds. FP32 values occupy 4 bytes and FP64 values 8, so a 256-bit register holds eight FP32 lanes but only four FP64 lanes, and FP64 moves twice as much data per element.

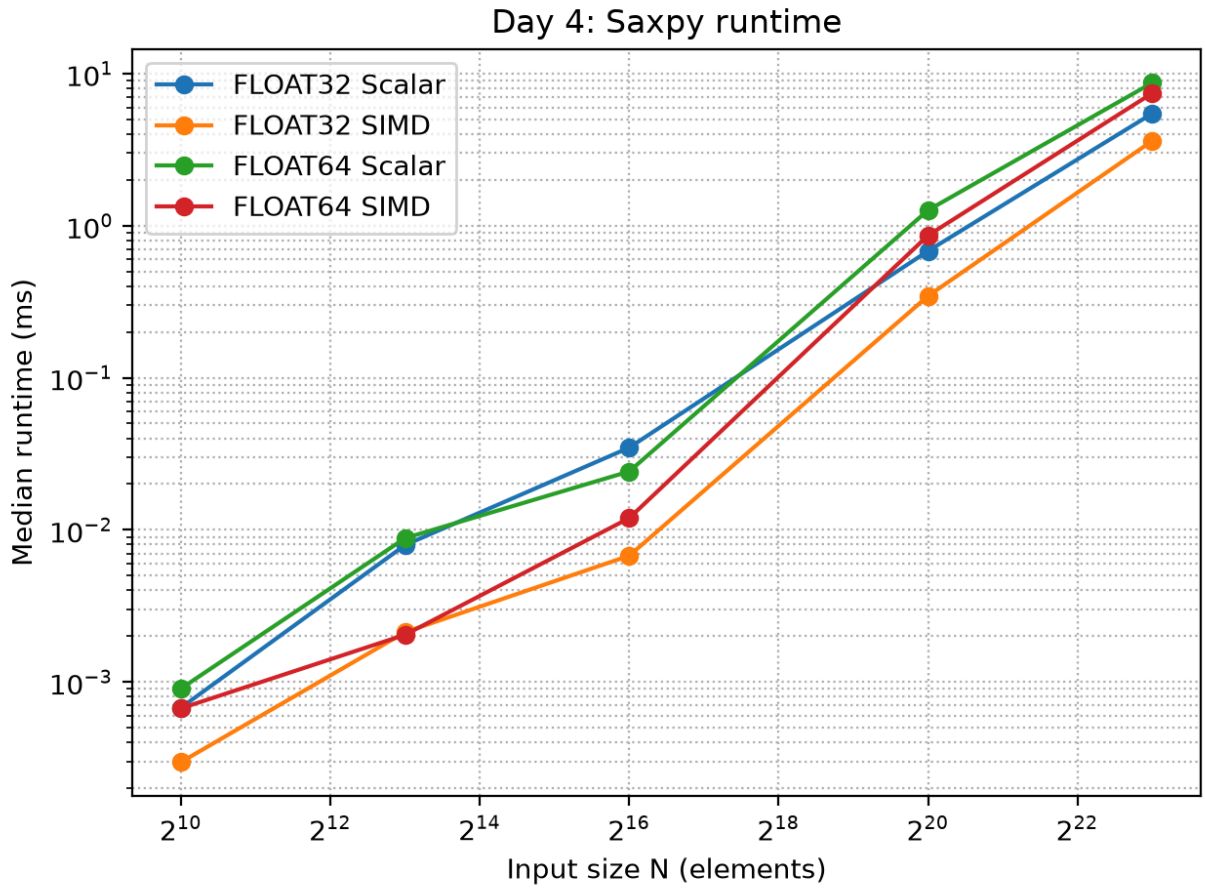
Multiply Runtime

Figure J. Elementwise multiplication runtime for float32 and float64, scalar and SIMD builds, from 2^{10} to 2^{23} elements (log axes). AVX2 processes eight float32 or four float64 values per instruction, and float64 doubles the bytes per element, halving arithmetic intensity from 0.083 to 0.042 FLOP/byte. Float64 therefore reaches cache limits sooner: at 2^{20} its SIMD speedup is 1.1× against 1.8× for float32. At 2^{23} both types reach about 25 GiB/s, and float64 takes 2.1 times as long, consistent with its doubled data movement.



Saxpy Runtime

Figure K. SAXPY runtime for float32 and float64, scalar and SIMD builds, from 2^{10} to 2^{23} elements (log axes). SAXPY ($y_i = a\,x_i + y_i$) performs two FLOPs per element, giving 0.167 FLOP/byte for float32 and 0.083 for float64, and AVX2 holds eight float32 lanes but only four float64 lanes. Float64 SIMD speedup peaks at 2.0× at 2^{16}, against 5.1× for float32. At 2^{23} both types reach about 25 GiB/s, and float64 takes 2.1 times as long with a 1.2× SIMD speedup. The equal SIMD runtimes at 2^{13} and inverted scalar results at 2^{16} are likely measurement variability.

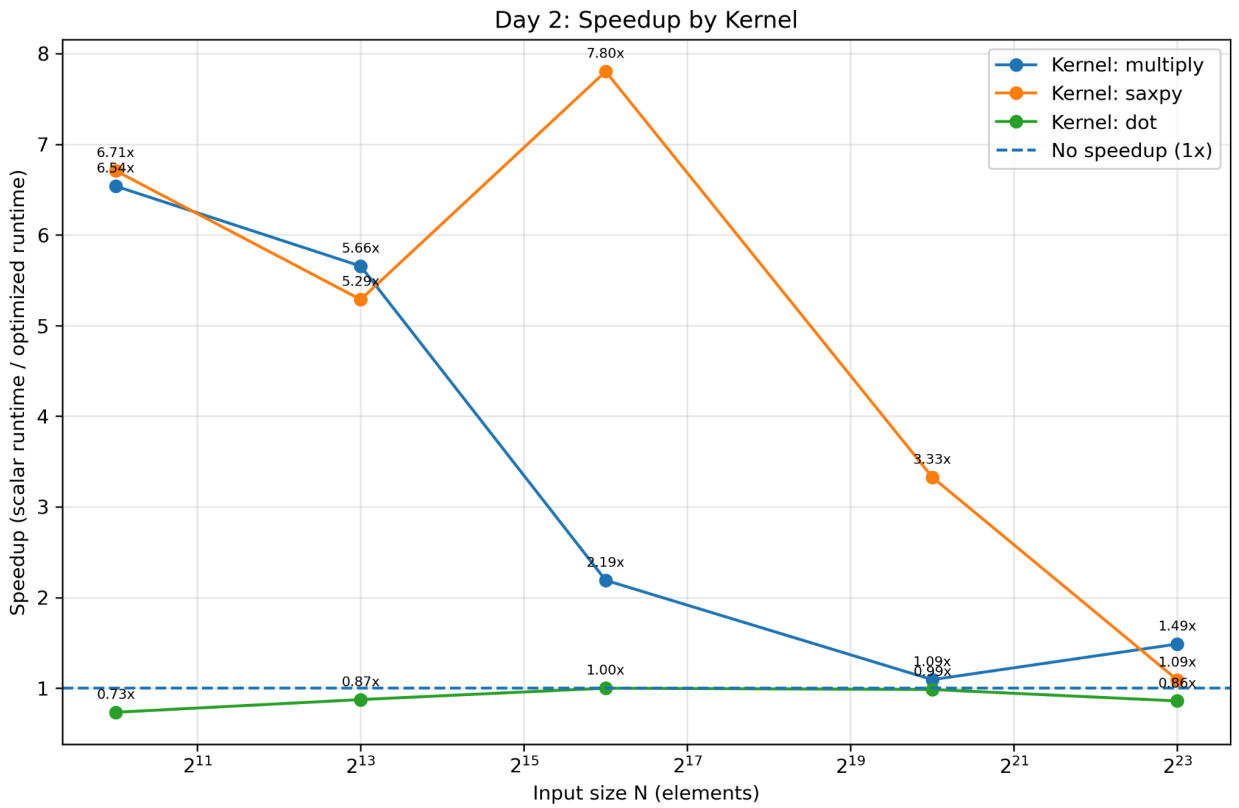


Vectorization Verification

This experiment checks whether the compiler actually generated SIMD instructions. Since -march=native also turns on other optimizations, a speedup on its own doesn't prove SIMD is responsible.

Speedup vs Size

Compiler reports and assembly should show packed floating-point instructions in the multiply and SAXPY loops. Dot product is the one to watch: strict floating-point rules stop the compiler from reordering the sum, which can keep it from vectorizing.



Alignment Runtime

Figure L. Day 2 speedup by kernel**.** Multiply and SAXPY reach much higher speedups than dot product at some sizes, consistent with vectorized elementwise loops and a poorly vectorized reduction. Compiler reports and assembly are needed to confirm this directly.

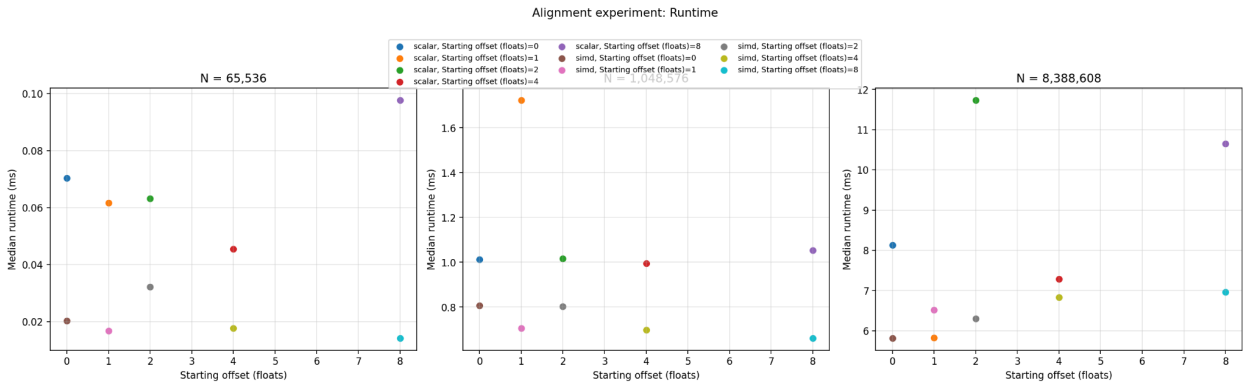


Figure M. Alignment experiment: scalar and SIMD runtime by starting offset.

Roofline Interpretation for One Kernel

The roofline model caps a kernel's performance at either memory bandwidth or peak compute, whichever it hits first. Where its arithmetic intensity falls relative to the ridge point, where the two limits meet, shows which one applies.

Triad Kernel

Triad performs two FLOPs per element (a multiply and an add) and nominally moves 12 bytes (two loads, one store), giving 0.167 FLOP/byte. The sloped roof is the highest observed logical bandwidth, about 16.5 GB/s, and the flat roof an estimated 998 GFLOP/s peak, meeting at a ridge point near 60 FLOP/byte. Triad sits far below the ridge, in the memory-bound region, with a bandwidth ceiling of about 2.75 GFLOP/s.

All four measured points lie near the bandwidth roof: about 2.75 (scalar) and 2.6 GFLOP/s (SIMD) at N = 16,777,216, and 1.85 and 2.25 GFLOP/s at N = 8,388,608. Memory throughput, not arithmetic capacity, limits these large inputs, so SIMD helps only slightly at the smaller size and not at all at the larger one.

A few caveats. The bandwidth roof is the best observed result, not a real DRAM measurement, so the scalar point at N = 16,777,216 sits on it by definition. The 12-byte count also ignores read-for-ownership traffic, so the real intensity is a bit lower. And the 998 GFLOP/s peak is a rough estimate that treats every core as identical, even though this chip mixes core types. None of this changes the conclusion: triad is memory-bound.

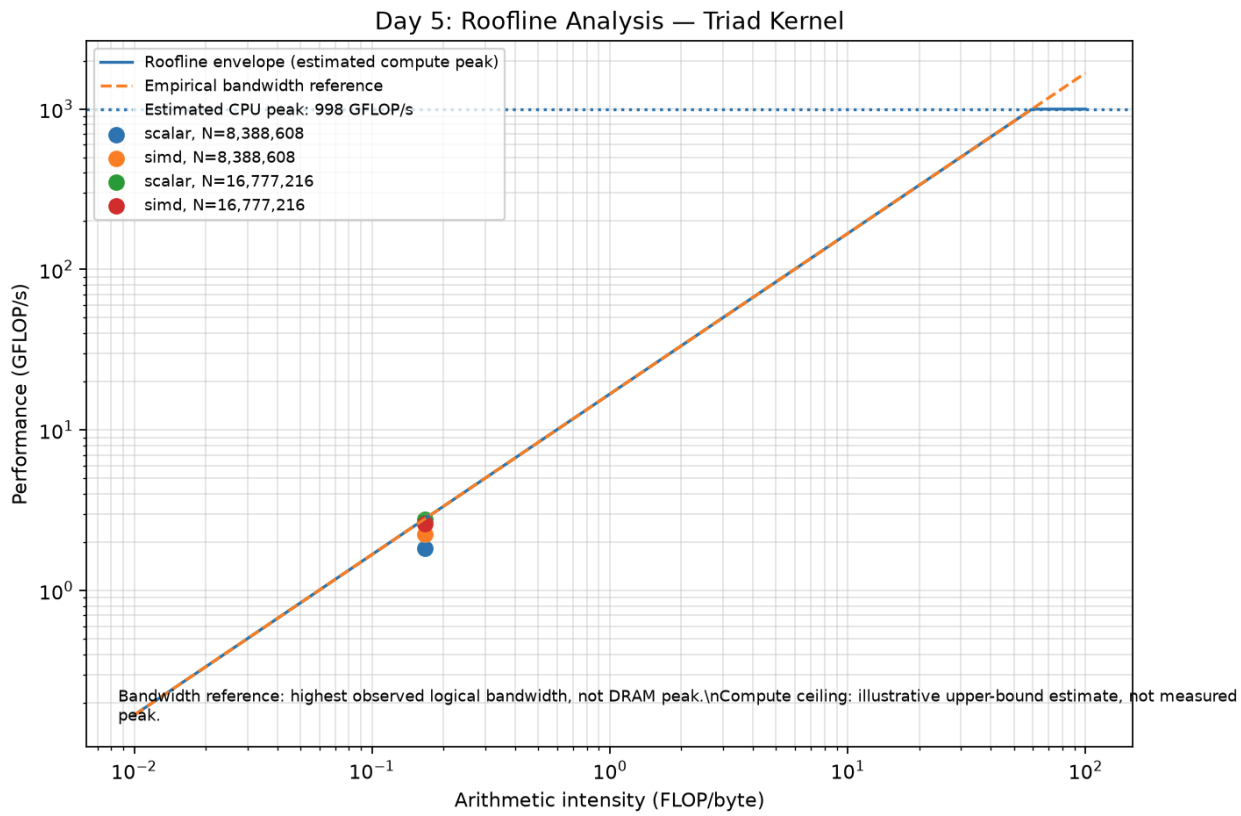


Figure N. Roofline analysis for the triad kernel.

Scalar vs SIMD runtime

Elementwise multiplication performs one FLOP per 12 bytes, or 0.083 FLOP/byte, half of triad's intensity. With the same 16.5 GB/s roof, its bandwidth ceiling is about 1.4 GFLOP/s.

Small inputs fit in cache, which offers far more bandwidth than DRAM, so cache-resident points can exceed the single DRAM roof. A cache-aware roofline would add separate L1, L2, and L3 ceilings.

Once inputs outgrow the cache, memory traffic takes over and the SIMD speedup shrinks, since vectorizing speeds up the math but not the data movement.

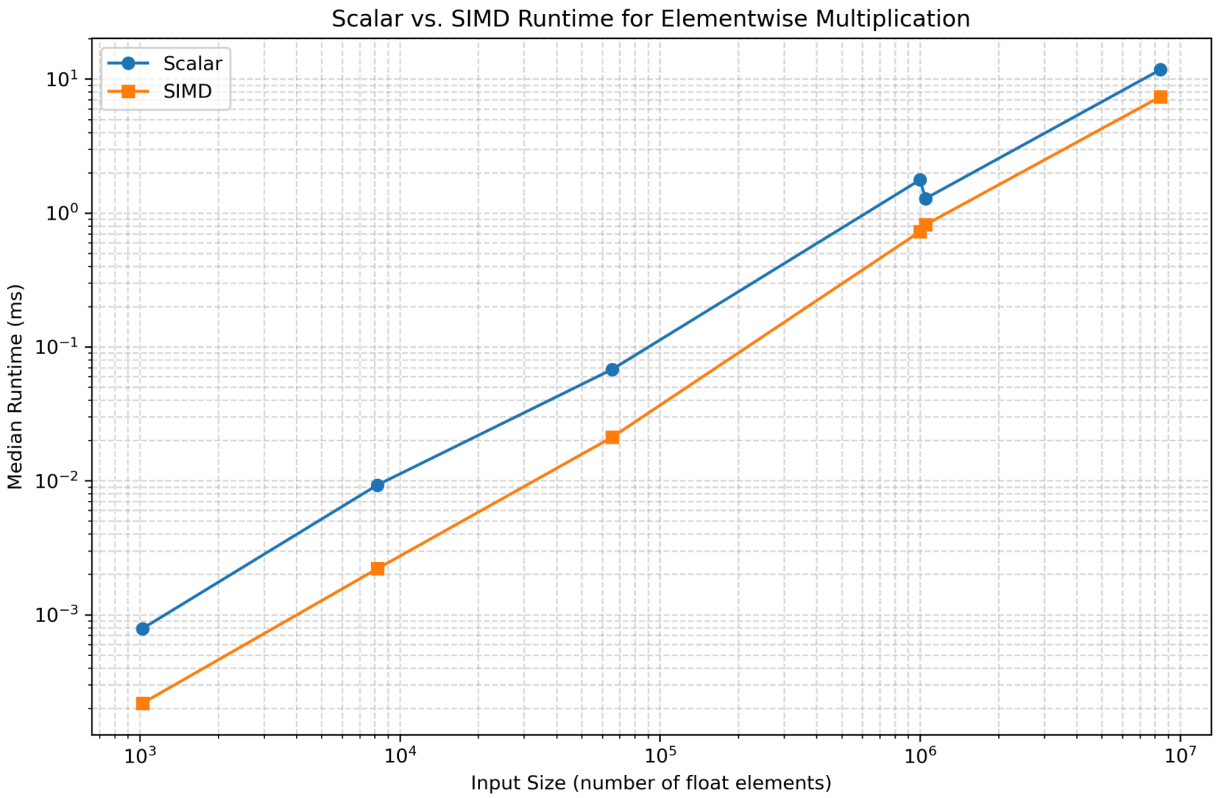


Figure O. Scalar vs. SIMD runtime for elementwise multiplication.

One Negative Example

Triad is a loop that vectorizes well but gains little, because two reads and one write per element give it low arithmetic intensity, leaving it limited by memory bandwidth.

SIMD was 1.22× faster at 8,388,608 elements and slightly slower at 16,777,216, showing that vectorization does not guarantee a speedup when bandwidth is the bottleneck.

Triad Kernel Achieved Performance

In Figure P, SIMD reaches 2.24 GFLOP/s against 1.84 for scalar at 8.39 million elements (22% faster), but 2.62 against 2.79 GFLOP/s at 16.78 million (6% slower).

Doing eight values per instruction doesn't make the whole loop eight times faster. Triad (A[i] = B[i] + sC[i]) does only two FLOPs for every two reads and one write, and both working sets are far bigger than the 12 MB L3, so it spends most of its time waiting on main memory. Faster math doesn't help much when the data can't arrive any faster.

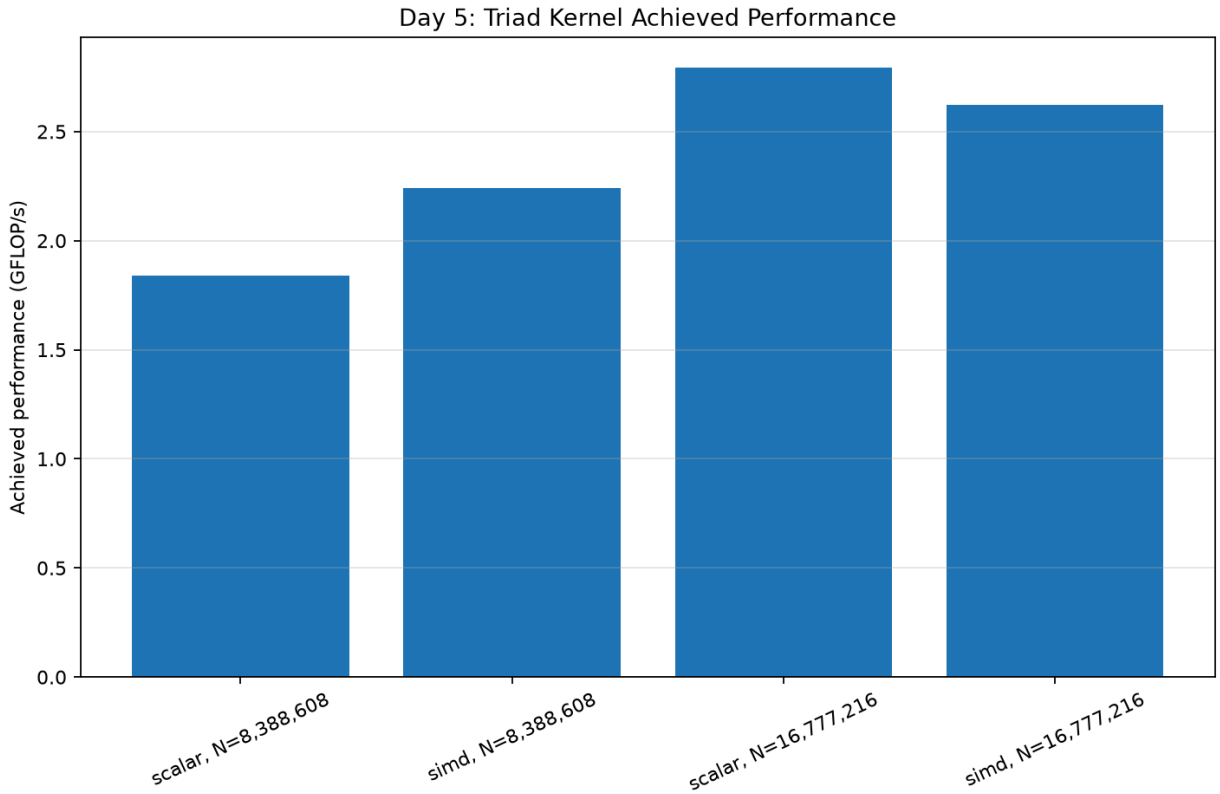


Figure P. Achieved performance for the triad kernel.

Roofline Analysis for Triad

Figure Q places triad at 0.167 FLOP/byte, far below the ridge point of about 60 FLOP/byte set by the 16.5 GB/s bandwidth roof and 998 GFLOP/s compute estimate, in the memory-bound region.

Its bandwidth ceiling is about 2.75 GFLOP/s, and the measured 1.84 to 2.79 GFLOP/s is consistent with data movement limiting performance. Both reference values are estimates, as noted in the roofline section.

SIMD pays off when a kernel does a lot of math per value it loads, or when its data fits in cache. Triad does neither.

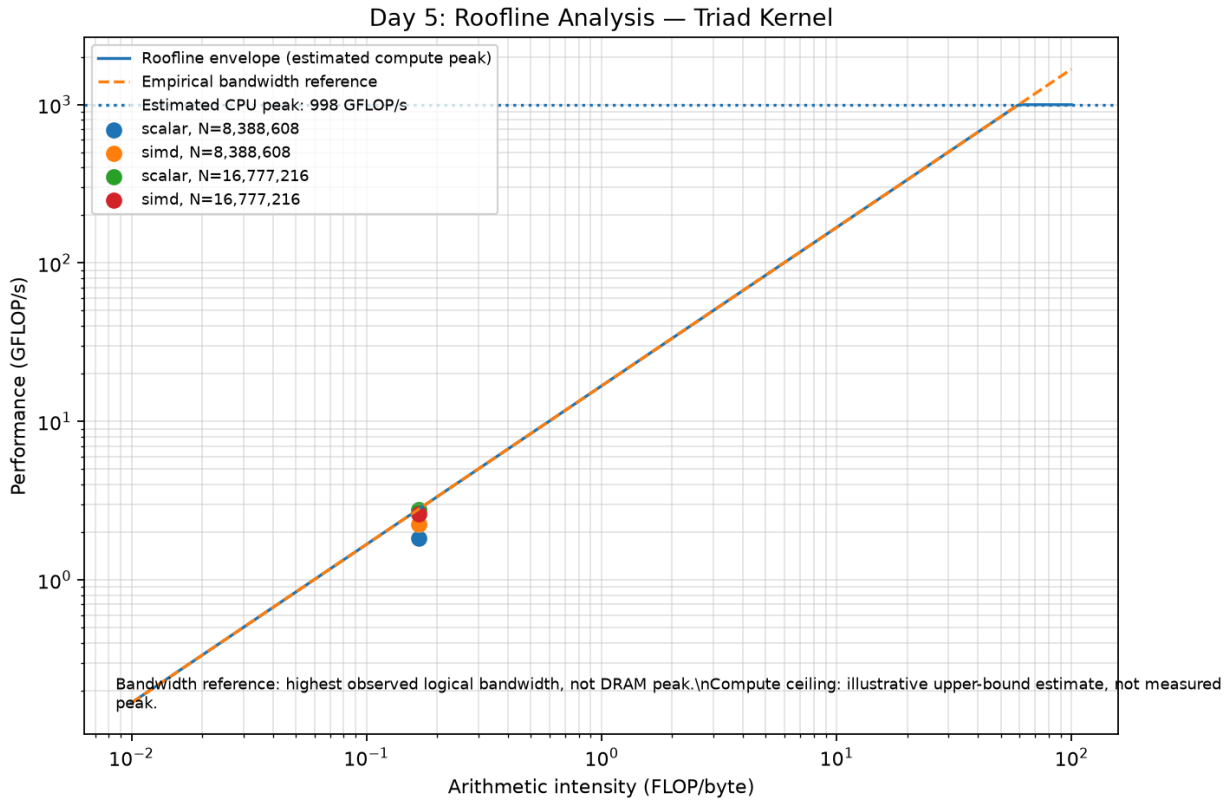


Figure Q. Roofline analysis for the triad kernel.

Conclusion

I compared scalar and compiler-optimized versions of elementwise multiplication, SAXPY, and dot product across working-set size, alignment, tail handling, stride, data type, and arithmetic intensity.

SIMD helped most when the data fit in cache and was read in order: multiply and SAXPY got close to 8× at some sizes. Dot product barely improved, alignment and tails made no clear difference, and large strides slowed everything down. Triad showed that even a loop that vectorizes well gains little once memory bandwidth is the limit.

The takeaway: vector width is only part of the story. Data locality, bandwidth, and arithmetic intensity matter just as much, so an 8-wide unit rarely means an 8× speedup.

Honest AI Usage

Claude was used to help me format this report and as a supplementary tool during this project to assist with writing plotting code for CSV datasets and to provide general coding

assistance when needed. This included help with processing collected CSV data and generating visualizations for analyzing the experimental results. The AI assistance was used to support the implementation and presentation of the experiments, while the project's measurements and resulting performance analysis were based on the collected data. AI-generated code and suggestions were reviewed and adapted as needed for the project. Claude was also used to edit and condense the report's wording, label the figures, and build the table of contents; I reviewed the edited text for accuracy.